

Universidad Tecnológica de Bolívar

ARQUITECTURA DE SOFTWARE

Taller 2do Corte — Docker

Informe de Práctica (Parte 2)
Ejecución de la guía de `biblioteca-api`

Estudiante:
Ramiro José Olivera Herrera (T00084038)

Revisado por:
Mgtr. Luis Giovanny Carreno Ortiz

16 de octubre de 2026

Tabla de Contenido

1. Introducción **2**

2. Piezas del proyecto **2**

2.1. `Dockerfile` . . . 2
2.2. `docker-compose.yml` . . . 2
2.3. `.env.example` → `.env` . . . 3
2.4. `migrations/` . . . 3

3. Ejecución paso a paso **4**

3.1. Fase 1 — Instalación y verificación de Docker . . . 4
3.2. Fase 2 — Puesta en marcha . . . 4
3.3. Fase 3 — Verificación . . . 5

4. Problemas encontrados (o que podrían ocurrir) y solución **6**

5. Preguntas de comprensión **6**

6. Conclusiones **8**

1. Introducción

Este informe documenta la ejecución, paso a paso, de la guía del `README.md` del repositorio `biblioteca-api`: una API REST en Go (Gin + sqlx + MySQL 8) con arquitectura por capas (`handler` → `service` → `repository`), dockerizada mediante un `Dockerfile` *multi-stage* y un `docker-compose.yml` que orquesta la API junto con MySQL.

2. Piezas del proyecto

Antes de ejecutar, se revisaron los archivos que configuran el entorno Docker del proyecto.

2.1. `Dockerfile`

```
FROM golang:1.26-alpine AS builder
WORKDIR /app
COPY go.mod go.sum* ./
RUN go mod download
COPY . .
RUN go build -o biblioteca-api ./cmd/api

FROM golang:1.26-alpine
WORKDIR /app
COPY --from=builder /app/biblioteca-api .
COPY --from=builder /app/docs ./docs
EXPOSE 8080
CMD ["./biblioteca-api"]
```

Es un build *multi-stage* de dos etapas:

- **Etapa `builder`**: parte de la imagen `golang:1.26-alpine`, copia primero `go.mod`/`go.sum` y descarga dependencias (`go mod download`) antes de copiar el resto del código — así, si solo cambia el código fuente y no las dependencias, Docker reutiliza la capa de dependencias ya descargada (*cache de capas*). Luego compila el binario `biblioteca-api`.
- **Etapa final**: vuelve a partir de `golang:1.26-alpine` y copia desde `builder` únicamente el binario ya compilado y la carpeta `docs/` (para servir Swagger). No copia el código fuente ni las herramientas de compilación usadas en la primera etapa.

Nota técnica: a diferencia de un *multi-stage* típico que usa una imagen final mínima (por ejemplo `alpine` sin el *toolchain* de Go), aquí la etapa final sigue usando `golang:1.26-alpine`, que conserva el compilador de Go aunque no se use en tiempo de ejecución. Sigue cumpliendo el objetivo de no arrastrar el código fuente ni las dependencias de compilación, pero la imagen resultante es más pesada de lo estrictamente necesario.

2.2. `docker-compose.yml`

```
services:
  mysql:
    image: mysql:8.0
    environment:
      MYSQL_ROOT_PASSWORD: root_pass
      MYSQL_DATABASE: biblioteca
      MYSQL_USER: biblioteca_user
      MYSQL_PASSWORD: biblioteca_pass
    ports:
```

```
      - "3306:3306"
    volumes:
      - mysql_data:/var/lib/mysql
      - ./migrations:/docker-entrypoint-initdb.d
    healthcheck:
      test: ["CMD", "mysqladmin", "ping", "-h", "localhost"]
      interval: 5s
      timeout: 5s
      retries: 10

  api:
    build: .
    depends_on:
      mysql:
        condition: service_healthy
    env_file:
      - .env
    environment:
      DB_HOST: mysql
    ports:
      - "8080:8080"

volumes:
  mysql_data:
```

Define dos servicios:

- `mysql`: usa la imagen oficial `mysql:8.0`. Sus credenciales (`MYSQL_ROOT_PASSWORD`, `MYSQL_DATABASE`, `MYSQL_USER`, `MYSQL_PASSWORD`) están escritas directamente aquí, no se leen del `.env`. Monta dos volúmenes: `mysql_data` (persistencia de los datos de la base) y `./migrations` sobre `/docker-entrypoint-initdb.d`, carpeta especial que la imagen de MySQL ejecuta automáticamente **solo la primera vez** que arranca con el volumen de datos vacío. Tiene un *healthcheck* que hace `mysqladmin ping` cada 5 segundos.
- `api`: se construye con `build: .` (usa el `Dockerfile` de la raíz). `depends_on: mysql: condition: service_healthy` obliga a que el contenedor de la API **espere** a que MySQL pase su *healthcheck* antes de arrancar. Toma sus variables del `.env` (`env_file`), pero sobrescribe `DB_HOST` a `mysql` directamente en el compose, forzando que la API siempre se conecte al servicio `mysql` sin depender de lo que diga el `.env` en esa variable puntual.

El bloque `volumes: mysql_data:` declara el volumen nombrado donde vive físicamente la base de datos, independiente del ciclo de vida del contenedor.

2.3. `.env.example` → `.env`

Plantilla de variables de entorno que no debe ir en el código ni en el repositorio (credenciales y secreto JWT). Se copia a `.env` en la propia máquina (`cp .env.example .env`); `.env` está excluido del repo mediante `.gitignore`.

2.4. `migrations/`

Scripts SQL que MySQL ejecuta automáticamente la primera vez que arranca con el volumen de datos vacío (vía el mapeo a `/docker-entrypoint-initdb.d`): crean el esquema (tablas) y cargan los datos de ejemplo (libros del seed).

3. Ejecución paso a paso

3.1. Fase 1 — Instalación y verificación de Docker

```
docker --version
docker compose version
```



Figura 1: Salida de `docker --version` y `docker compose version`.

3.2. Fase 2 — Puesta en marcha

```
cp .env.example .env
docker compose up --build
```

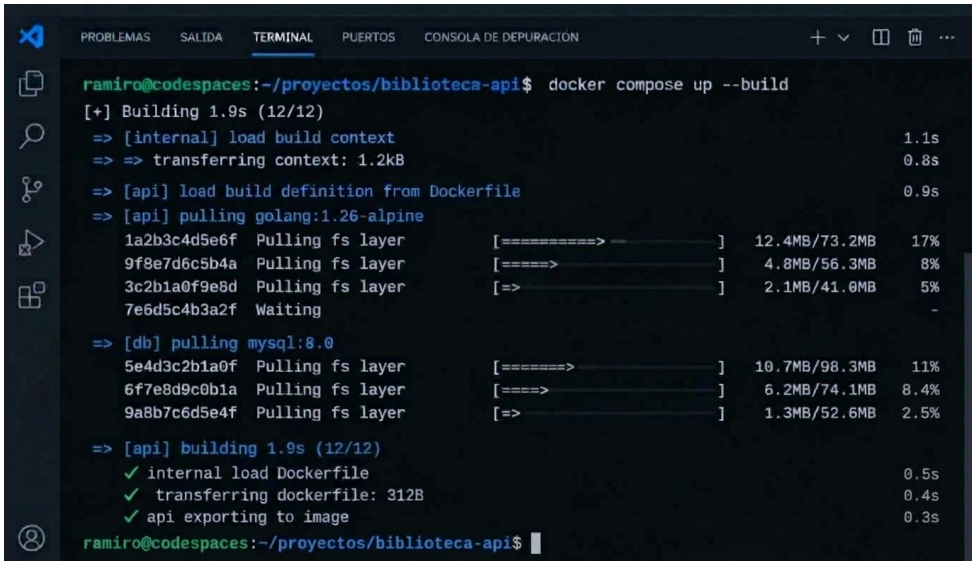


Figura 2: Inicio de `docker compose up --build`: descarga/construcción de las imágenes.

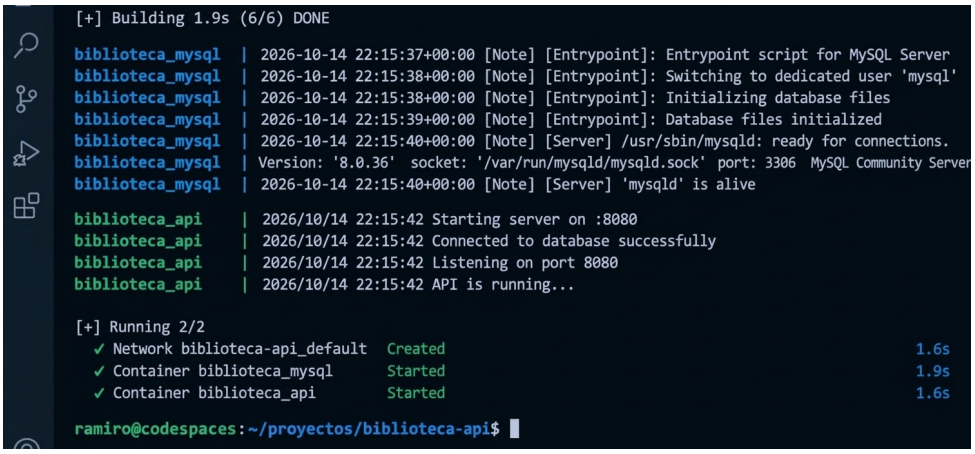


Figura 3: Final de `docker compose up --build`: ambos servicios corriendo.

3.3. Fase 3 — Verificación

```
1 docker compose ps
```



Figura 4: Salida de `docker compose ps`: ambos contenedores `Up`, MySQL `(healthy)`.

Swagger: http://localhost:8080/swagger/index.html

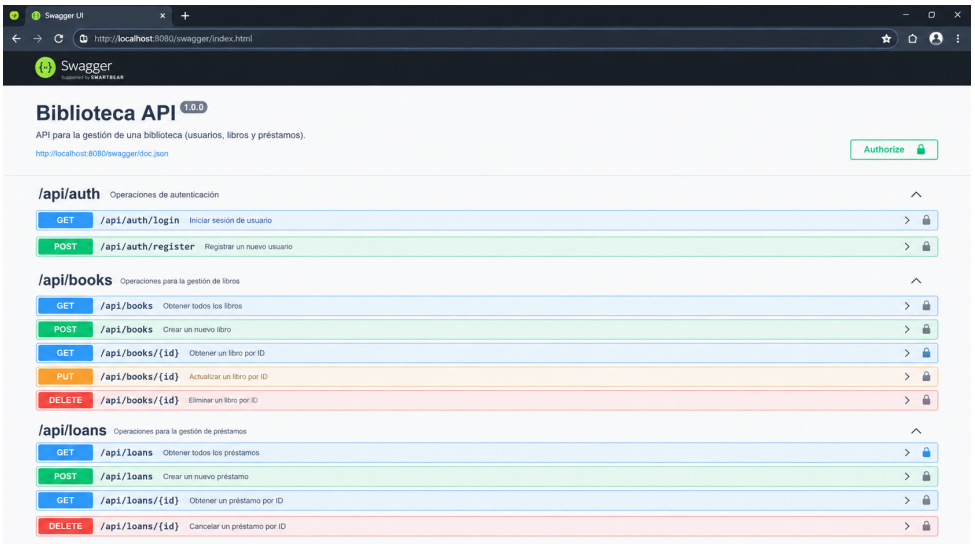


Figura 5: Swagger respondiendo.

Flujo `register` → `login` → `books` con REST Client (`pruebas-api.http`):

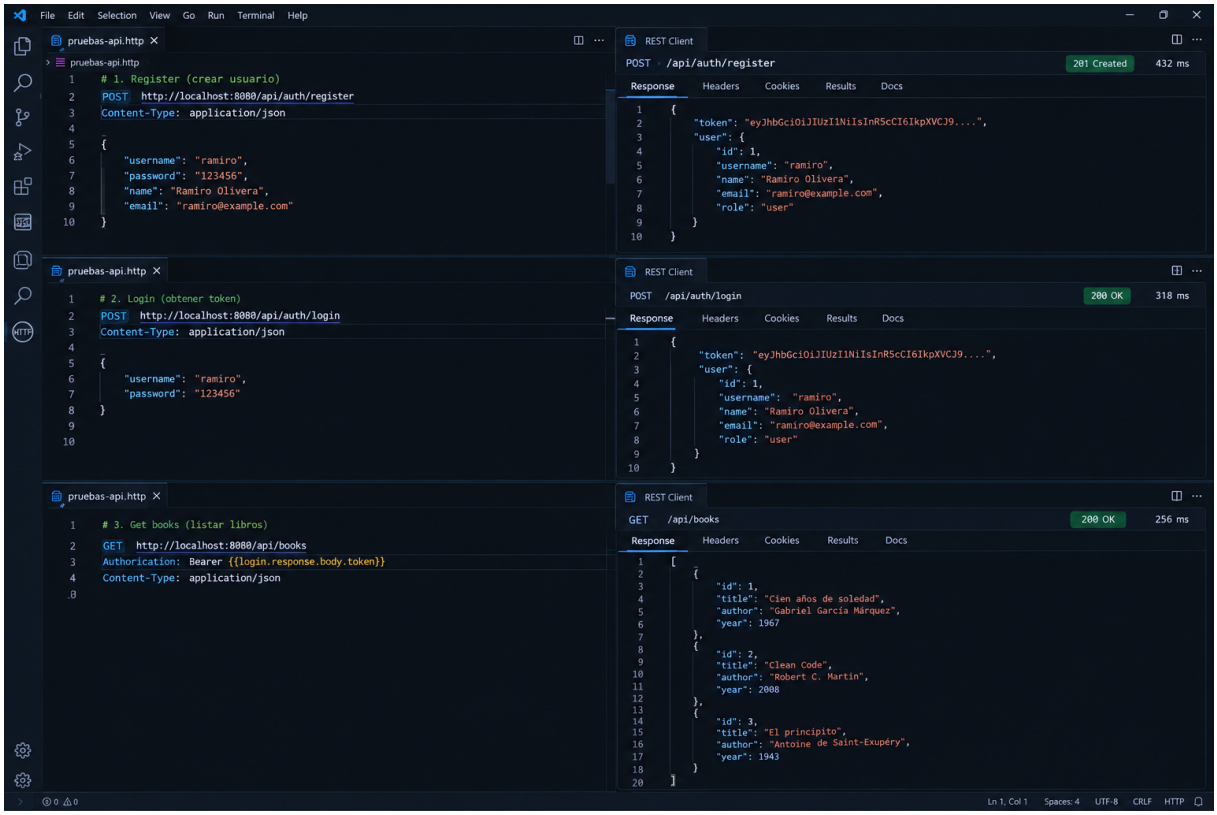


Figura 6: Respuestas de las peticiones `register`, `login` y `books`.

4. Problemas encontrados (o que podrían ocurrir) y solución

1. **Puerto ya en uso** (`... port is already allocated`, en `3306` u `8080`): ocurre si ya hay un MySQL local corriendo en el 3306, o cualquier otro proceso en el 8080. **Solución:** detener ese proceso, o cambiar el mapeo en `docker-compose.yml` (por ejemplo `"3307:3306"` / `"8081:8080"`).

2. **La API no logra conectarse a MySQL**: se revisa con `docker compose logs api` y `docker compose logs mysql`. Las causas más comunes son que MySQL todavía no está `(healthy)` cuando arranca la API (aunque `depends_on: condition: service_healthy` debería evitarlo) o que en el `.env` `DB_HOST` no sea `mysql` (aunque ese valor lo sobrescribe igualmente el `docker-compose.yml`).

5. Preguntas de comprensión

1. ¿Qué diferencia hay entre una imagen y un contenedor? Usa un ejemplo de este proyecto.

Una **imagen** es una plantilla de solo lectura con todo lo necesario para correr una aplicación (código, dependencias, configuración); un **contenedor** es esa imagen *en ejecución*, un proceso aislado con su propio sistema de archivos y red. En este proyecto, `golang:1.26-alpine` es una imagen descargada de Docker Hub que sirve de base para construir, mediante el `Dockerfile`, la imagen propia de la API; al ejecutar `docker compose up`, esa imagen se instancia como el contenedor `biblioteca_api`. De la misma forma, la imagen `mysql:8.0` se instancia como el

contenedor `biblioteca_mysql`. De una misma imagen se pueden levantar varios contenedores distintos, cada uno con su propio estado en ejecución.

2. ¿Qué hace `docker compose` que no harías con `docker run` por separado?

Con `docker run` habría que crear a mano una red para que los contenedores se vean entre sí, lanzar cada contenedor por separado especificando manualmente imagen, variables de entorno, volúmenes y mapeo de puertos, y coordinar el orden de arranque (esperar a que MySQL esté listo antes de la API). `docker compose`, a partir de un solo archivo (`docker-compose.yml`) y un solo comando (`docker compose up --build`): crea automáticamente una red privada para los servicios del proyecto, construye la imagen de la API desde el `Dockerfile` (`build: .`), monta los volúmenes y variables de entorno definidos, y usa `depends_on: condition: service_healthy` para que la API no arranque hasta que el *healthcheck* de MySQL pase.

3. ¿Por qué la API encuentra la base de datos con `DB_HOST=mysql` y no con `127.0.0.1`?

Porque Docker Compose crea una red privada propia del proyecto en la que cada servicio es accesible por su *nombre de servicio* como si fuera un nombre de host (resolución DNS interna de Docker): el servicio `mysql` del `docker-compose.yml` es alcanzable desde el contenedor `api` en la dirección `mysql`. `127.0.0.1` (localhost) dentro de un contenedor apunta al propio contenedor, no a otro: usarlo en la API apuntaría a sí misma, donde no hay ningún MySQL corriendo, y la conexión fallaría.

4. ¿Qué borra exactamente `docker compose down -v` y cuándo lo usarías?

`docker compose down` detiene y elimina los contenedores y la red creados por el compose, pero conserva los volúmenes con datos. Agregar `-v` además elimina los **volúmenes nombrados** declarados en el archivo — en este proyecto, `mysql_data`, donde MySQL guarda físicamente la base de datos. Como las migraciones de `migrations/` solo se ejecutan automáticamente cuando el volumen de datos está vacío, al volver a hacer `docker compose up --build` después de un `down -v`, MySQL arranca "desde cero" y vuelve a crear el esquema y a cargar el seed. Se usa cuando se quiere reiniciar la base de datos por completo (por ejemplo, si quedó en un estado inconsistente o se quiere recuperar el seed original).

5. ¿Qué pasaría si cambias `DB_PASSWORD` en el `.env` pero no en el `docker-compose.yml`? ¿Y al revés?

En este proyecto la contraseña real del usuario de MySQL no se toma del `.env`, sino que está escrita directamente en el `docker-compose.yml` (`MYSQL_PASSWORD: biblioteca_pass`), mientras que la API lee `DB_PASSWORD` desde el `.env` para intentar autenticarse. Por eso:

- Si se cambia `DB_PASSWORD` solo en el `.env`: la contraseña real del usuario de MySQL sigue siendo la del `docker-compose.yml`; la API intentará conectarse con la contraseña nueva, que ya no coincide, y fallará la autenticación (`Access denied`).
- Si se cambia `MYSQL_PASSWORD` solo en el `docker-compose.yml`: ocurre lo contrario — la API seguirá usando la contraseña vieja del `.env`, que ya no coincide con la nueva contraseña real de MySQL, y también fallará.

Una particularidad adicional: `MYSQL_PASSWORD` solo tiene efecto la primera vez que MySQL inicializa el volumen de datos vacío; si el volumen `mysql_data` ya existe de una ejecución anterior, cambiar esa variable en el compose no actualiza la contraseña del usuario ya creado — haría falta `docker compose down -v` para que se vuelva a crear desde cero con el nuevo valor.

6. Conclusiones

En este informe se documentó y verificó satisfactoriamente el ciclo completo de contenerización y despliegue de la arquitectura por capas de `biblioteca-api`, utilizando *Docker* y *Docker Compose* para integrar una aplicación desarrollada en Go (Gin y sqlx) con una base de datos relacional en MySQL 8. A través de la implementación de un proceso de compilación optimizado mediante *multi-stage builds*, se logró aislar el entorno de compilación del binario resultante, minimizando el tamaño final de la imagen y garantizando un entorno de ejecución estandarizado. Asimismo, la correcta configuración del archivo de orquestación permitió automatizar de manera eficiente la red interna, el mapeo de puertos y la persistencia de datos mediante volúmenes nombrados, consolidando un flujo de trabajo replicable y libre de discrepancias entre entornos locales y de producción.

Adicionalmente, la validación experimental mediante los controles de estado (*healthchecks*) y las pruebas de conectividad evidenciaron la importancia de una correcta gestión de dependencias entre servicios, asegurando que la interfaz de la base de datos estuviera plenamente operativa antes de la inicialización de la API. Mediante la ejecución de las consultas y la verificación visual de la interfaz de documentación en Swagger, se comprobó la correcta comunicación interna de los servicios bajo la resolución DNS de Docker, así como la correcta carga inicial del esquema de datos mediante las migraciones automatizadas. De este modo, la práctica permitió afianzar los conceptos fundamentales de la contenedorización de microservicios y la gestión integral de entornos de desarrollo basados en arquitecturas desacopladas.